অধ্যায় ২

Variables, Data Types এবং Memory

শেখার লক্ষ্য

এই অধ্যায় শেষ করার পর আপনি পারবেন:

- `let`, `const` এবং `var`-এর পার্থক্য ব্যাখ্যা করে সঠিকটি বেছে নিতে।
- JavaScript-এর Primitive Type-গুলো চিনতে এবং `typeof` দিয়ে যাচাই করতে।
- `undefined` এবং `null`-এর পার্থক্য বুঝে সঠিক জায়গায় ব্যবহার করতে।
- String থেকে Number-এ এবং অন্য Type-এ নিরাপদে রূপান্তর করতে।
- `===` এবং `==`-এর পার্থক্য ব্যাখ্যা করতে।
- Primitive মান এবং Object Reference মেমোরিতে কীভাবে আলাদা আচরণ করে, তা বুঝতে।
- Type-সংক্রান্ত সাধারণ Bug খুঁজে বের করতে।
- একটি সম্পূর্ণ **Student Result Calculator** তৈরি ও পরীক্ষা করতে।

পূর্বশর্ত

- অধ্যায় ১ সম্পন্ন করা: `console.log()`, String, Function-এর প্রাথমিক পরিচয় এবং `if`।
- Node.js এবং VS Code Install করা আছে, এবং Terminal থেকে `node file.js` চালাতে পারেন।
- `chapter-02` নামে একটি নতুন Folder, যেখানে এই অধ্যায়ের File-গুলো রাখবেন।

এই অধ্যায় কেন গুরুত্বপূর্ণ

একটি Program আসলে তথ্য নিয়ে কাজ করে: নাম, নম্বর, দাম, তারিখ, হ্যাঁ অথবা না। এই তথ্য কোথায় রাখা হবে, কোন ধরনের তথ্য কীভাবে আচরণ করে এবং এক ধরন থেকে আরেক ধরনে গেলে কী ঘটে—এই তিনটি প্রশ্নের উত্তরই এই অধ্যায়ের বিষয়।

বিষয়গুলো সহজ মনে হলেও বাস্তব Program-এর বহু Bug এখান থেকেই আসে। একটি Form থেকে আসা `'80'` এবং `'90'` যোগ করলে `170` না হয়ে `8090` হয়ে যায়। একটি তালিকা Copy করে বদলালে মূল তালিকাটিও বদলে যায়। ফাঁকা ঘরের মান শূন্য ধরে নেওয়া হয়। প্রতিটি সমস্যার কারণ এই অধ্যায়ে পরিষ্কার হবে।

অধ্যায়ের শেষে আপনি একটি ফলাফল হিসাবের Program লিখবেন, যেখানে Text হিসেবে আসা নম্বর যাচাই করে সংখ্যায় রূপান্তর করা হয়, তারপর মোট, গড় এবং Grade বের করা হয়।

২.১ Variable: মানের নাম

অধ্যায় ১-এ আপনি দেখেছেন, **Variable** হলো একটি নাম, যার সঙ্গে একটি মান যুক্ত থাকে। মান যুক্ত করার কাজটিকে **Assignment** বলে, আর `=` চিহ্নটি Assignment Operator।

variables.js · JavaScript — চলবে: Browser Console ও Node.js

```
const studentName = 'Sara';
let totalMarks = 240;

console.log(studentName);
console.log(totalMarks);

totalMarks = 250;
console.log(totalMarks);
```

Output

```
Sara
240
250
```

প্রথম দুই লাইনে দুটি Variable তৈরি করা হয়েছে। Variable তৈরি করাকে **Declaration** বলে। ষষ্ঠ লাইনে `totalMarks` -এ নতুন মান রাখা হয়েছে; একে **Reassignment** বলে।

Variable-এর নাম রাখার নিয়ম

নিয়ম	সঠিক	ভুল
অক্ষর, সংখ্যা, `_` এবং `$` ব্যবহার করা যায়	`total1`, `_count`	`total-marks`
সংখ্যা দিয়ে শুরু করা যায় না	`mark1`	`1mark`
ফাঁকা জায়গা থাকতে পারে না	`studentName`	`student name`
ভাষার নিজস্ব শব্দ ব্যবহার করা যায় না	`result`	`const`, `return`
বড় ও ছোট হাতের অক্ষর আলাদা	`total` এবং `Total` দুটি ভিন্ন নাম	—

নিয়ম মানলেই নাম ভালো হয় না। `x` বা `data` নাম থেকে কিছু বোঝা যায় না; `totalMarks` বা `studentName` পড়লেই বোঝা যায় ভেতরে কী আছে। এই বইয়ে camelCase রীতি ব্যবহার করা হয়েছে।

২.২ let, const এবং var

JavaScript-এ Variable তৈরি করার তিনটি শব্দ আছে। পার্থক্যগুলো এক নজরে:

বৈশিষ্ট্য	`const`	`let`	`var`
নতুন মান রাখা যায়	না	হ্যাঁ	হ্যাঁ
তৈরির সময় মান দেওয়া বাধ্যতামূলক	হ্যাঁ	না	না
একই নামে আবার তৈরি করা যায়	না	না	হ্যাঁ
কোথায় কার্যকর	যে `{ }`-এর ভেতরে তৈরি	যে `{ }`-এর ভেতরে তৈরি	পুরো Function জুড়ে

`const` : যে নামের মান বদলাবে না

const-error.js · JavaScript চলবে: Node.js

```
const passMark = 33;
passMark = 40;
```

Error Message

```
TypeError: Assignment to constant variable.
```

`const` দিয়ে তৈরি নামে দ্বিতীয়বার মান রাখতে গেলে JavaScript Program থামিয়ে দেয়। এটি অসুবিধা নয়, বরং সুরক্ষা: যে মান বদলানোর কথা নয়, সেটি ভুল করে বদলে গেলে সঙ্গে সঙ্গে জানা যায়।

`let` : যে নামের মান বদলাবে

গণনা, যোগফল বা অবস্থা—যে মান Program চলার সময় বদলায়, তার জন্য `let`।

Block এবং Scope

দ্বিতীয় বন্ধনী `{ }`-এর ভেতরের অংশকে **Block** বলে। কোনো Variable Program-এর যে অংশ থেকে ব্যবহার করা যায়, সেই অংশকে তার **Scope** বলে। `let` এবং `const` দিয়ে তৈরি Variable শুধু নিজের Block-এর ভেতরে কার্যকর।

block-scope.js · JavaScript চলবে: Browser Console ও Node.js

```
let count = 1;

if (true) {
  let count = 2;
  console.log(count);
}

console.log(count);
```

Output

```
2
1
```

Block-এর ভেতরের `count` একটি সম্পূর্ণ আলাদা Variable। Block শেষ হলে সেটি আর থাকে না, এবং বাইরের `count` অপরিবর্তিত থাকে। Scope নিয়ে বিস্তারিত আলোচনা অধ্যায় ৫-এ আছে।

`var` : পুরোনো পদ্ধতি

`var` JavaScript-এর শুরুর দিনের পদ্ধতি। পুরোনো Code পড়ার জন্য এটি চেনা দরকার, কিন্তু এর দুটি আচরণ সহজেই Bug তৈরি করে।

var-problems.js · JavaScript চলবে: Browser Console ও Node.js

```
var city = 'Dhaka';
var city = 'Sylhet';
console.log(city);

if (true) {
  var leaked = 'Block-এর ভেতরে তৈরি';
}

console.log(leaked);
```

Output

```
Sylhet
Block-এর ভেতরে তৈরি
```

প্রথমত, একই নামে দুবার `var` লিখলে কোনো Error হয় না; দ্বিতীয়টি নীরবে প্রথমটিকে মুছে দেয়। বড় File-এ এভাবে অন্যের Variable নষ্ট হয়ে যেতে পারে। দ্বিতীয়ত, `var` Block মানে না: `if`-এর ভেতরে তৈরি `leaked` বাইরে থেকেও পাওয়া যাচ্ছে।

> **মনে রাখুন:** প্রথমে `const` ব্যবহার করুন। মান বদলানোর প্রয়োজন হলে তবেই `let`। নতুন Code-এ `var` ব্যবহার করবেন না।

তৈরি করার আগে ব্যবহার

`let` বা `const` দিয়ে তৈরি Variable তার Declaration-এর লাইনের আগে ব্যবহার করা যায় না।

before-declaration.js · JavaScript চলবে: Node.js

```
console.log(score);
let score = 95;
```

Error Message

```
ReferenceError: Cannot access 'score' before initialization
```

বার্তাটি স্পষ্ট: `score` আছে, কিন্তু এখনো প্রস্তুত নয়। সমাধান হলো Variable আগে তৈরি করা, পরে ব্যবহার করা।

২.৩ Data Type এবং typeof

প্রতিটি মানের একটি ধরন আছে, যাকে **Data Type** বলে। ধরন ঠিক করে দেয় মানটি নিয়ে কী করা যাবে: সংখ্যা গুণ করা যায়, লেখা যায় না।

JavaScript-এ সাতটি **Primitive Type** আছে। Primitive মানে মৌলিক: এগুলো একটি করে সরল মান, যা ভেঙে আর ছোট করা যায় না।

Type	কী রাখে	উদাহরণ
`string`	লেখা	`'Dhaka'`
`number`	পূর্ণসংখ্যা ও দশমিক সংখ্যা	`42`, `3.14`
`boolean`	সত্য অথবা মিথ্যা	`true`, `false`
`undefined`	এখনো কোনো মান দেওয়া হয়নি	`undefined`
`null`	ইচ্ছাকৃতভাবে "কিছু নেই"	`null`
`bigint`	অত্যন্ত বড় পূর্ণসংখ্যা	`9007199254740993n`
`symbol`	অনন্য পরিচয়চিহ্ন	`Symbol('id')`

প্রথম পাঁচটি প্রতিদিন লাগবে। `bigint` ও `symbol` বিশেষ ক্ষেত্রে ব্যবহৃত হয়; এখন শুধু নাম জেনে রাখুন। Primitive নয় এমন সব মান **Object**, যা অনুচ্ছেদ ২.৯-এ আসবে।

কোনো মানের Type জানতে `typeof` ব্যবহার করুন।

typeof.js · JavaScript — চলবে: Browser Console ও Node.js

```
console.log(typeof 'Dhaka');
console.log(typeof 42);
console.log(typeof 3.14);
console.log(typeof true);
console.log(typeof undefined);
console.log(typeof 10n);
console.log(typeof Symbol('id'));
console.log(typeof null);
```

Output

```
string
number
number
boolean
undefined
bigint
symbol
object
```

দুটি বিষয় লক্ষ করুন। পূর্ণসংখ্যা ও দশমিক সংখ্যা—দুটোরই Type `number`; JavaScript-এ এদের আলাদা Type নেই। আর `typeof null` দেখায় `object`, যদিও `null` একটি Primitive। এটি ভাষার একটি পুরোনো ত্রুটি, যা পুরোনো Website ভেঙে যাওয়ার আশঙ্কায় আর ঠিক করা হয়নি। কোনো মান `null` কি না জানতে `typeof` নয়, `=== null` ব্যবহার করুন।

২.৪ Number-এর আচরণ

Computer দশমিক সংখ্যা নির্দিষ্ট পরিমাণ জায়গায় রাখে, তাই কিছু সংখ্যা হুবহু রাখা সম্ভব হয় না। ফলে কয়েকটি ফলাফল প্রথমবার অবাক করে।

numbers.js · JavaScript — চলবে: Browser Console ও Node.js

```
console.log(0.1 + 0.2);
console.log(10 / 3);
console.log(5 / 0);
console.log(Number('abc'));
console.log(typeof NaN);
console.log(NaN === NaN);
console.log(Number.isNaN(Number('abc')));
```

Output

```
0.30000000000000004
3.3333333333333335
Infinity
NaN
number
false
true
```

- `0.1 + 0.2` ঠিক `0.3` হয় না। এটি JavaScript-এর ত্রুটি নয়; প্রায় সব Programming Language একই পদ্ধতিতে দশমিক সংখ্যা রাখে।
- শূন্য দিয়ে ভাগ করলে Error হয় না; ফল হয় `Infinity`।
- `NaN`-এর পূর্ণরূপ Not a Number। যে হিসাবের অর্থপূর্ণ সংখ্যা-ফল নেই, তার ফল `NaN`। অদ্ভুত শোনালেও এর Type `number`।
- `NaN` নিজের সমানও নয়, তাই `=== NaN` দিয়ে একে ধরা যায় না। সব সময় `Number.isNaN()` ব্যবহার করুন।

> **সতর্কতা:** টাকার হিসাবে দশমিক সংখ্যা ব্যবহার করবেন না। `0.1 + 0.2`-এর মতো সামান্য ভুল হাজার হাজার হিসাবে জমে বড় হয়ে যায়। টাকাকে পয়সায় (পূর্ণসংখ্যায়) রেখে হিসাব করুন, দেখানোর সময় টাকায় রূপান্তর করুন।

দশমিকের পরে নির্দিষ্ট সংখ্যক ঘর দেখাতে `toFixed()` ব্যবহার করা হয়। এটি একটি String ফেরত দেয়, তাই শুধু দেখানোর জন্য ব্যবহার করুন, হিসাবের জন্য নয়।

to-fixed.js · JavaScript — চলবে: Browser Console ও Node.js

```
const average = 220 / 3;

console.log(average);
console.log(average.toFixed(2));
console.log(typeof average.toFixed(2));
```

Output

```
73.33333333333333
73.33
string
```

২.৫ undefined এবং null

দুটোই "মান নেই" বোঝায়, কিন্তু কারণ আলাদা।

- **`undefined`** — JavaScript নিজে দেয়। Variable তৈরি হয়েছে, কিন্তু কোনো মান রাখা হয়নি।

- `null` — Programmer ইচ্ছা করে দেন। অর্থ: "এখানে জেনেশুনেই কিছু রাখা হয়নি।"

undefined-null.js · JavaScript চলবে: Browser Console ও Node.js

```
let email;
const middleName = null;

console.log(email);
console.log(middleName);
console.log(email === undefined);
console.log(middleName === null);
```

Output

```
undefined
null
true
true
```

একটি নিবন্ধন Form-এর কথা ভাবুন। Email-এর ঘরটি এখনো পূরণ করা হয়নি: অবস্থাটি `undefined`। ব্যবহারকারী জানিয়েছেন তাঁর মধ্য নাম নেই: অবস্থাটি `null`। প্রথমটি "জানা নেই", দ্বিতীয়টি "জানা আছে যে নেই"।

নিজে কখনো কোনো Variable-এ `undefined` রাখবেন না। "কিছু নেই" বোঝাতে হলে `null` ব্যবহার করুন; তাহলে `undefined` দেখলেই বোঝা যাবে কোথাও মান দিতে ভুল হয়েছে।

২.৬ Dynamic Typing

JavaScript-এ Type থাকে মানের, Variable-এর নয়। একই Variable-এ প্রথমে সংখ্যা, পরে লেখা রাখা যায়। এই বৈশিষ্ট্যকে **Dynamic Typing** বলে।

dynamic-typing.js · JavaScript চলবে: Browser Console ও Node.js

```
let value = 42;
console.log(typeof value);

value = 'বিয়াল্লিশ';
console.log(typeof value);

value = true;
console.log(typeof value);
```

Output

```
number
string
boolean
```

এতে Code লেখা সহজ হয়, কিন্তু দায়িত্বও বাড়ে: কোনো Variable-এ এই মুহূর্তে কোন Type-এর মান আছে, তা JavaScript আপনাকে আগে থেকে জানায় না। তাই একটি Variable-কে একটিই কাজে এবং একটিই Type-এ ব্যবহার করা ভালো অভ্যাস। `totalMarks` -এ সব সময় সংখ্যাই রাখুন।

২.৭ Type Conversion

এক Type-এর মানকে অন্য Type-এ বদলানোকে **Type Conversion** বলে। এটি দুইভাবে ঘটে: আপনি নিজে করলে **Explicit Conversion**, আর JavaScript নিজে থেকে করলে **Implicit Conversion** বা Coercion।

Explicit Conversion: নিজে রূপান্তর করা

explicit-conversion.js · JavaScript — চলবে: Browser Console ও Node.js

```
console.log(Number('42'));
console.log(Number('  7  '));
console.log(Number('12px'));
console.log(Number(''));
console.log(Number(true));
console.log(Number(null));
console.log(Number(undefined));
console.log(typeof String(42));
console.log(Boolean(0));
console.log(Boolean(''));
console.log(Boolean('0'));
```

Output

```
42
7
NaN
0
1
0
NaN
string
false
false
true
```

রূপান্তর	নিয়ম
`Number(text)`	পুরো লেখাটি সংখ্যা হলে সেই সংখ্যা; আগে-পরের ফাঁকা জায়গা উপেক্ষিত; অন্যথায় `NaN`
`Number('')`	`0` — ফাঁকা String শূন্য হয়ে যায়
`String(value)`	মানটির লিখিত রূপ

রূপান্তর	নিয়ম
`Boolean(value)`	`0`, `''`, `null`, `undefined`, `NaN` হলে `false`; বাকি সব `true`

`Boolean('0')`-এর ফল `true`, কারণ `'0'` একটি ফাঁকা নয় এমন String। কোন মান সত্য আর কোনটি মিথ্যা হিসেবে গণ্য হয়, তা অধ্যায় ৩-এ বিস্তারিত আসবে।

> **সতর্কতা:** `Number('')` এর ফল `0`। Form-এর ফাঁকা ঘরকে সরাসরি `Number()` দিলে সেটি শূন্য নম্বর হয়ে যাবে, আর Program বুঝতেই পারবে না যে ঘরটি পূরণ করা হয়নি। রূপান্তরের আগে সব সময় ফাঁকা কি না যাচাই করুন।

parseInt এবং parseFloat

`Number()` কঠোর: লেখার কোনো অংশ সংখ্যা না হলে পুরো ফলই `NaN`। কখনো কখনো লেখার শুরুর সংখ্যাটুকু দরকার হয়, যেমন `'12px'` থেকে `12`। সে কাজে `parseInt()` এবং `parseFloat()` ব্যবহার করা হয়।

parse-int.js · JavaScript — চলবে: Browser Console ও Node.js

```
console.log(Number('12px'));
console.log(parseInt('12px', 10));
console.log(parseFloat('3.5kg'));
console.log(parseInt('80.9', 10));
console.log(parseInt('abc', 10));
```

Output

```
NaN
12
3.5
80
NaN
```

`parseInt()` বাঁ দিক থেকে পড়ে এবং প্রথম যে অক্ষরটি সংখ্যার অংশ নয়, সেখানে থেমে যায়। দশমিক বিন্দুও তার কাছে থামার জায়গা, তাই `'80.9'` থেকে `80` পাওয়া যায়; এটি সংখ্যাকে নিকটতম মানে আনে না, শুধু কেটে ফেলে। দ্বিতীয় মান `10` জানায় যে সংখ্যাটি দশভিত্তিক; এটি সব সময় লিখুন।

এই নমনীয়তা যাচাইয়ের কাজে বিপজ্জনক। কেউ নম্বরের ঘরে `'80abc'` লিখলে `parseInt()` সেটিকে নীরবে `80` হিসেবে গ্রহণ করবে। তাই ব্যবহারকারীর দেওয়া মান যাচাই করতে `Number()` ব্যবহার করুন, আর `parseInt()` রাখুন সেই ক্ষেত্রের জন্য, যেখানে লেখার শেষে একক থাকাই স্বাভাবিক।

Implicit Conversion: JavaScript যখন নিজে রূপান্তর করে

implicit-conversion.js · JavaScript চলবে: Browser Console ও Node.js

```
console.log('5' + 2);
console.log('5' - 2);
console.log('5' * '2');
console.log(1 + 2 + '3');
console.log('1' + 2 + 3);
```

Output

```
52
3
10
33
123
```

নিয়মটি `+` চিহ্নকে ঘিরে। `+`-এর দুটি কাজ: সংখ্যা যোগ করা এবং String জোড়া লাগানো। দুই পাশের যেকোনো একটি String হলে JavaScript অন্যটিকেও String বানিয়ে জোড়া লাগায়। `-` এবং `*`-এর এমন দ্বৈত ভূমিকা নেই, তাই সেখানে String-কে সংখ্যায় রূপান্তর করা হয়।

শেষ দুই লাইন বাঁ থেকে ডানে হিসাব হয়। `1 + 2 + '3'`-তে প্রথমে `1 + 2` হয় `3`, তারপর `3 + '3'` হয় `'33'`। `'1' + 2 + 3`-তে প্রথম ধাপেই String তৈরি হয়ে যায়: `'12'`, তারপর `'123'`।

Implicit Conversion-এর ওপর নির্ভর করবেন না। যেখানে সংখ্যা দরকার, সেখানে `Number()` দিয়ে নিজে রূপান্তর করুন।

রূপান্তরের ফল যাচাই করা

validate-number.js · JavaScript চলবে: Browser Console ও Node.js

```
console.log(Number.isNaN(Number('abc')));
console.log(Number.isInteger(42));
console.log(Number.isInteger(42.5));
console.log(Number.isInteger(Number('80')));
```

Output

```
true
true
false
true
```

`Number.isNaN()` জানায় রূপান্তর ব্যর্থ হয়েছে কি না। `Number.isInteger()` জানায় মানটি পূর্ণসংখ্যা কি না; `NaN` এবং দশমিক সংখ্যা—দুটোর জন্যই এটি `false` দেয়। Project-এ এই দুটিই কাজে লাগবে।

‘‘

২.৮ সমতা যাচাই: === এবং ==

দুটি মান সমান কি না যাচাইয়ের দুটি Operator আছে।

- `===` (Strict Equality) — Type এবং মান দুটোই মিলতে হবে।
- `==` (Loose Equality) — Type আলাদা হলে আগে রূপান্তর করে, তারপর তুলনা করে।

equality.js · JavaScript — চলবে: Browser Console ও Node.js

```
console.log(0 === '0');
console.log(0 == '0');
console.log('' == 0);
console.log(null == undefined);
console.log(null === undefined);
console.log(5 !== '5');
```

Output

```
false
true
true
true
false
true
```

`==`-এর রূপান্তরের নিয়মগুলো মনে রাখা কঠিন এবং ফলাফল প্রায়ই অপ্রত্যাশিত: ফাঁকা String আর শূন্য "সমান" হয়ে যায়। তাই এই বইয়ে সব সময় `===` ব্যবহার করা হয়েছে। "সমান নয়" বোঝাতে `!==` ব্যবহার করুন; শেষ লাইনে সংখ্যা `5` এবং String `'5'` আলাদা Type, তাই ফল `true`।

২.৯ Memory Model: মান এবং Reference

এ পর্যন্ত দেখা সব মান ছিল Primitive। এবার **Object**-এর পরিচয়: Object হলো একাধিক তথ্যকে একসঙ্গে রাখার একটি মান। দ্বিতীয় বন্ধনীর ভেতরে নাম ও মানের জোড়া লিখে Object তৈরি করা হয়, এবং ডট (`.`) দিয়ে ভেতরের তথ্য পড়া হয়। Object নিয়ে পূর্ণ আলোচনা অধ্যায় ৭-এ; এখানে শুধু মেমোরির আচরণটি দেখা হবে।

Primitive: মান Copy হয়

primitive-copy.js · JavaScript — চলবে: Browser Console ও Node.js

```javascript
let a = 5;
let b = a;

b = 9;

console.log(a);
console.log(b);
```

Output

```
5
9
```

`let b = a` লেখার সময় `5` মানটির একটি Copy `b`-তে রাখা হয়েছে। এরপর `a` এবং `b`-র মধ্যে কোনো সম্পর্ক নেই।

Object: ঠিকানা Copy হয়

object-reference.js · JavaScript — চলবে: Browser Console ও Node.js

```javascript
const first = { name: 'Nila' };
const second = first;

second.name = 'Nila Islam';

console.log(first.name);
console.log(first === second);

const third = { name: 'Nila Islam' };
console.log(first === third);
```

Output

```
Nila Islam
true
false
```

Object তৈরি হলে সেটি মেমোরির কোনো এক জায়গায় থাকে, আর Variable-এ রাখা হয় সেই জায়গার ঠিকানা। এই ঠিকানাকে **Reference** বলে। `const second = first` Object-টি Copy করে না; শুধু ঠিকানাটি Copy করে। ফলে দুটি নাম একই Object দেখায়, এবং একটির মাধ্যমে বদলালে অন্যটিতেও পরিবর্তন দেখা যায়।

	Primitive	Object
Variable-এ যা থাকে	মানটি নিজে	মানটির ঠিকানা (Reference)

	Primitive	Object
`b = a` করলে	মান Copy হয়	ঠিকানা Copy হয়; Object একটিই থাকে
`===` যা তুলনা করে	মান	ঠিকানা

শেষ সারিটি `first === third`-এর ফল ব্যাখ্যা করে। দুটি Object-এর ভেতরের তথ্য একই, কিন্তু এরা মেমোরির দুটি আলাদা জায়গায় আছে। ঠিকানা আলাদা, তাই ফল `false`।

`const` এবং Object

const-object.js · JavaScript — চলবে: Browser Console ও Node.js

```
const profile = { name: 'Rafi' };

profile.name = 'Rafi Ahmed';

console.log(profile.name);
```

Output

```
Rafi Ahmed
```

`const` থাকা সত্ত্বেও কোনো Error হয়নি। কারণ `const` শুধু নিশ্চিত করে যে `profile` নামটি সব সময় একই Object-এর ঠিকানা ধরে রাখবে। Object-এর ভেতরের তথ্য বদলানো নিষেধ নয়। নিষেধ হলো `profile = { ... }` লিখে নামটিকে অন্য Object-এ সরিয়ে নেওয়া।

Function-এ মান পাঠালে কী ঘটে

একই নিয়ম Function Call-এর সময়ও খাটে। Function-এ Primitive পাঠালে মানের Copy যায়; Object পাঠালে ঠিকানার Copy যায়।

pass-to-function.js · JavaScript — চলবে: Browser Console ও Node.js

```
function addBonus(mark) {
  mark = mark + 5;
  return mark;
}

const mark = 70;
const updated = addBonus(mark);
console.log(mark);
console.log(updated);

function promote(student) {
  student.level = 2;
}
```

```
const student = { name: 'Mina', level: 1 };
promote(student);
console.log(student.level);
```

Output

```
70
75
2
```

`addBonus`-এর ভেতরের `mark` বাইরের `mark`-এর একটি Copy; ভেতরে বদলালেও বাইরের মান `70`-ই থাকে। কিন্তু `promote` যে ঠিকানা পেয়েছে, সেটি বাইরের Object-টিরই ঠিকানা। তাই Function-এর ভেতরে করা পরিবর্তন বাইরেও দেখা যায়।

এই আচরণ জানা না থাকলে খুঁজে পাওয়া কঠিন Bug তৈরি হয়: একটি Function "শুধু হিসাব করার জন্য" একটি Object নিল, আর অজান্তে সেটি বদলে দিল। অভ্যাস হিসেবে, Function-এর ভেতরে পাওয়া Object বদলাবেন না; প্রয়োজন হলে নতুন Object তৈরি করে ফেরত দিন।

আলাদা Copy তৈরি করা

সত্যিকারের আলাদা Object দরকার হলে তিনটি ডট (`...`) ব্যবহার করে নতুন Object তৈরি করা যায়।

object-copy.js · JavaScript — চলবে: Browser Console ও Node.js

```
const original = { name: 'Arif', city: 'Dhaka' };
const copy = { ...original };

copy.city = 'Khulna';

console.log(original.city);
console.log(copy.city);
console.log(original === copy);
```

Output

```
Dhaka
Khulna
false
```

`{ ...original }` একটি নতুন Object তৈরি করে এবং পুরোনোটির তথ্যগুলো তাতে বসায়। এই Copy শুধু প্রথম স্তরের জন্য: Object-এর ভেতরে আরেকটি Object থাকলে সেটির ঠিকানাই Copy হয়। বিষয়টি অধ্যায় ৭-এ আবার আসবে।

> **নোট:** যে Object-এর দিকে আর কোনো Variable নির্দেশ করে না, JavaScript সেটির মেমোরি নিজে থেকেই ফিরিয়ে নেয়। এই ব্যবস্থার নাম Garbage Collection। আপনাকে নিজে মেমোরি মুক্ত করতে হয় না।

২.১০ সাধারণ ভুল

ভুল	কী দেখা যায়	সমাধান
String হিসেবে আসা সংখ্যা যোগ করা	`'80' + '90'` হয় `8090`	আগে `Number()` দিয়ে রূপান্তর করুন
ফাঁকা ঘরকে সরাসরি `Number()` দেওয়া	ফাঁকা ঘর `0` হয়ে যায়	রূপান্তরের আগে `=== ''` যাচাই করুন
`=== NaN` দিয়ে যাচাই	সব সময় `false`	`Number.isNaN()` ব্যবহার করুন
`const`-এ নতুন মান রাখা	`TypeError: Assignment to constant variable.`	মান বদলাতে হলে `let` ব্যবহার করুন
Declaration-এর আগে ব্যবহার	`ReferenceError: Cannot access ... before initialization`	Variable আগে তৈরি করুন
`==` ব্যবহার	`'' == 0` হয় `true`	`===` ব্যবহার করুন
Object "Copy" করতে `=` ব্যবহার	মূল Object-ও বদলে যায়	`{ ...object }` দিয়ে নতুন Object তৈরি করুন
`typeof` দিয়ে `null` যাচাই	ফল আসে `object`	`=== null` ব্যবহার করুন
`toFixed()`-এর ফল দিয়ে হিসাব	সংখ্যা জোড়া লেগে যায়	হিসাব শেষ করে তবেই `toFixed()` দিন

২.১১ Debugging নির্দেশিকা

Type-সংক্রান্ত Bug-এ সাধারণত কোনো Error Message আসে না; Program চলে, কিন্তু ফল ভুল হয়। তাই এখানে মূল কৌশল হলো মান এবং তার Type একসঙ্গে দেখা।

debug-types.js · JavaScript — চলবে: Browser Console ও Node.js

```
const bangla = '80';
const english = '90';

console.log(bangla + english);
console.log('bangla:', bangla, typeof bangla);

const total = Number(bangla) + Number(english);
console.log('total:', total, typeof total);
```

Output

```
8090
bangla: 80 string
total: 170 number
```

`console.log()`-এ কমা দিয়ে একাধিক মান দেওয়া যায়। প্রথমে একটি Label, তারপর মান, তারপর `typeof`—এই তিনটি একসঙ্গে দেখলে সমস্যাটি পরিষ্কার হয়: `80` দেখতে সংখ্যার মতো, কিন্তু Type `string`।

ফল ভুল হলে এই ক্রমে যাচাই করুন:

১. সন্দেহজনক প্রতিটি Variable-এর মান এবং `typeof` একসঙ্গে দেখুন।

২. ফল `NaN` হলে পেছনের দিকে যান: কোন রূপান্তর বা হিসাবটি প্রথম `NaN` তৈরি করেছে?

৩. দুটি সংখ্যা জোড়া লেগে গেলে খুঁজুন কোনটি String হিসেবে এসেছে।

৪. ফাঁকা ঘর শূন্য হয়ে গেলে দেখুন `Number()`-এর আগে ফাঁকা যাচাই আছে কি না।

৫. কোনো Object অপ্রত্যাশিতভাবে বদলালে খুঁজুন আর কোন Variable একই Object-এর Reference ধরে আছে।

৬. `ReferenceError` এলে দেখুন Variable-টি ব্যবহারের আগে এবং একই Block-এ তৈরি হয়েছে কি না।

২.১২ পেশাদার অভ্যাস

- **`const` দিয়ে শুরু করুন।** কোন মান বদলায় আর কোনটি বদলায় না, তা Code পড়েই বোঝা যায়।
- **একটি Variable, একটি উদ্দেশ্য, একটি Type।** একই নামে কখনো সংখ্যা, কখনো লেখা রাখবেন না।
- **বাইরে থেকে আসা সব তথ্যকে String ধরে নিন।** Form, File বা Network থেকে আসা মান যাচাই করে তবেই রূপান্তর করুন।
- **নিজে রূপান্তর করুন।** `Number()`, `String()` স্পষ্টভাবে লিখুন; Implicit Conversion-এর ওপর ভরসা করবেন না।
- **সব সময় `===` এবং `!==`।**
- **"নেই" বোঝাতে `null`।** `undefined` যেন শুধু ভুলের সংকেত হয়।
- **বিশেষ সংখ্যাকে নাম দিন।** Code-এর মাঝখানে `33` না লিখে `const PASS_MARK = 33;` লিখুন। যে মান পুরো Program-এ স্থির, তার নাম বড় হাতের অক্ষরে লেখা একটি প্রচলিত রীতি।
- **টাকার হিসাব পূর্ণসংখ্যায়।**

২.১৩ Project: Student Result Calculator

এবার একটি Program লিখবেন, যা একজন শিক্ষার্থীর তিনটি বিষয়ের নম্বর থেকে মোট, গড়, Grade এবং উত্তীর্ণ হওয়ার অবস্থা বের করে। নম্বরগুলো ইচ্ছা করেই String হিসেবে দেওয়া হবে, কারণ বাস্তবে Form বা File থেকে তথ্য এভাবেই আসে।

Project-এর চাহিদা

- শিক্ষার্থীর নাম এবং বাংলা, ইংরেজি ও গণিতের নম্বর String হিসেবে নেওয়া হবে।
- প্রতিটি নম্বর ০ থেকে ১০০-এর মধ্যে পূর্ণসংখ্যা হতে হবে। ফাঁকা, দশমিক বা সীমার বাইরের নম্বর গ্রহণ করা হবে না।
- মোট নম্বর, দুই দশমিক ঘরে গড় এবং Grade দেখাতে হবে।
- যেকোনো একটি বিষয়ে ৩৩-এর কম পেলে ফল হবে অনুত্তীর্ণ এবং Grade `F`।
- কোনো নম্বর ভুল হলে হিসাবের বদলে কোন বিষয়ের নম্বর ভুল, তা জানাতে হবে।

এখানে ব্যবহৃত Grade-এর সীমাগুলো শেখার জন্য একটি উদাহরণ; এটি কোনো শিক্ষা বোর্ডের সরকারি নিয়ম নয়।

File-এর কাঠামো

Folder-এর কাঠামো

```
student-result-calculator/
├── result.js
├── README.md
└── tests/
    └── result.test.js
```

`result.js` File-টি আপনি নিজে লিখবেন। `README.md` এবং Test File এই অধ্যায়ের Companion Source Code-এর `project/student-result-calculator/` Folder-এ আছে।

result.js

File-টি ১০৪ লাইনের, তাই তিন ধাপে দেখানো হলো। তিনটি অংশ পরপর একই File-এ লিখুন; `// ...` চিহ্নিত লাইনগুলো লিখতে হবে না, সেগুলো শুধু বোঝায় যে File-এর আরও অংশ আছে।

ধাপ ১: Text থেকে নিরাপদে নম্বর তৈরি

result.js · JavaScript নির্বাচিত অংশ (Selected Excerpt) চলবে: Node.js

```javascript
'use strict';

const PASS_MARK = 33;

// Text থেকে নম্বর তৈরি করে। নম্বর গ্রহণযোগ্য না হলে null ফেরত দেয়।
function toMark(text) {
  const clean = text.trim();

  if (clean === '') {
    return null;
  }

  const mark = Number(clean);

  if (Number.isInteger(mark) === false) {
```

```
    return null;
  }

  if (mark < 0) {
    return null;
  }

  if (mark > 100) {
    return null;
  }

  return mark;
}

// ...
```

`toMark` এই অধ্যায়ের মূল শিক্ষাগুলো এক জায়গায় আনে। প্রথমে ফাঁকা কি না যাচাই, কারণ `Number('')` শূন্য দেয়। তারপর রূপান্তর। `Number.isInteger(mark) === false` একসঙ্গে দুটি সমস্যা ধরে: `'abc'` থেকে আসা `NaN` এবং `'80.5'`-এর মতো দশমিক সংখ্যা। শেষে সীমা যাচাই; `<` মানে "ছোট" এবং `>` মানে "বড়" (তুলনার সব Operator অধ্যায় ৩-এ আছে)। কোনো যাচাইয়ে আটকে গেলে Function `null` ফেরত দেয়—অর্থাৎ "গ্রহণযোগ্য নম্বর নেই"।

ধাপ ২: গড় থেকে Grade

result.js · JavaScript নির্বাচিত অংশ (Selected Excerpt) চলবে: Node.js

```
// ...

// গড় নম্বর থেকে Grade ঠিক করে।
function getGrade(average) {
  if (average >= 80) {
    return 'A+';
  }

  if (average >= 70) {
    return 'A';
  }

  if (average >= 60) {
    return 'A-';
  }

  if (average >= 50) {
    return 'B';
  }

  if (average >= 40) {
    return 'C';
  }
```

```
  if (average >= PASS_MARK) {
    return 'D';
  }

  return 'F';
}

// ...
```

`>=` মানে "বড় অথবা সমান"। যাচাইগুলো বড় থেকে ছোট ক্রমে সাজানো, আর `return` চলামাত্র Function শেষ হয়। তাই গড় `75` হলে প্রথম শর্ত মেলে না, দ্বিতীয়টি মেলে, এবং `'A'` ফেরত দিয়ে Function সেখানেই থামে।

ধাপ ৩: পুরো ফলাফল

result.js · JavaScript নির্বাচিত অংশ (Selected Excerpt) চলবে: Node.js

```
// ...

// নাম ও তিনটি নম্বর (Text) থেকে ফলাফলের একটি বাক্য তৈরি করে।
function calculateResult(name, banglaText, englishText, mathText) {
  const bangla = toMark(banglaText);
  const english = toMark(englishText);
  const math = toMark(mathText);

  if (bangla === null) {
    return 'বাংলার নম্বর সঠিক নয়।';
  }

  if (english === null) {
    return 'ইংরেজির নম্বর সঠিক নয়।';
  }

  if (math === null) {
    return 'গণিতের নম্বর সঠিক নয়।';
  }

  const total = bangla + english + math;
  const average = total / 3;
  let grade = getGrade(average);
  let status = 'উত্তীর্ণ';

  if (bangla < PASS_MARK) {
    grade = 'F';
    status = 'অনুত্তীর্ণ';
  }

  if (english < PASS_MARK) {
    grade = 'F';
```

```
    status = 'অনুত্তীর্ণ';
  }

  if (math < PASS_MARK) {
    grade = 'F';
    status = 'অনুত্তীর্ণ';
  }

  const shown = average.toFixed(2);

  return `${name.trim()}: মোট ${total}, গড় ${shown}, Grade ${grade}, ${status}`;
}

console.log(calculateResult('Sara', '80', '90', '70'));
console.log(calculateResult('Rafi', '100', '100', '20'));
console.log(calculateResult('Mina', '75', '', '60'));
```

Output

```
Sara: মোট 240, গড় 80.00, Grade A+, উত্তীর্ণ
Rafi: মোট 220, গড় 73.33, Grade F, অনুত্তীর্ণ
ইংরেজির নম্বর সঠিক নয়।
```

Code-এর ধাপে ধাপে ব্যাখ্যা

- **রূপান্তর আগে, হিসাব পরে।** তিনটি নম্বরই প্রথমে `toMark` দিয়ে যাচাই ও রূপান্তর করা হয়। এরপরের পুরো হিসাবে শুধু `number` Type-এর মান থাকে, তাই `+` কখনো String জোড়া লাগাবে না।
- **`=== null` দিয়ে ব্যর্থতা ধরা।** `toMark` ব্যর্থ হলে `null` দেয়। শূন্য একটি বৈধ নম্বর, তাই "নম্বর নেই" বোঝাতে `0` ব্যবহার করা যেত না; এখানেই `null`-এর প্রয়োজন।
- **`const` এবং `let`।** `total` ও `average` একবার হিসাব হয়ে আর বদলায় না, তাই `const`। `grade` ও `status` পরে বদলাতে পারে, তাই `let`।
- **কোনো বিষয়ে অনুত্তীর্ণ হলে।** গড় যত ভালোই হোক, যেকোনো একটি নম্বর `PASS_MARK`-এর নিচে হলে `grade` ও `status` বদলে দেওয়া হয়। Rafi-র গড় ৭৩.৩৩, কিন্তু গণিতে ২০ পাওয়ায় Grade `F`।
- **`toFixed(2)` সবার শেষে।** হিসাব শেষ হওয়ার পর শুধু দেখানোর জন্য গড়কে String-এ রূপান্তর করা হয়েছে।

তিনটি `if` Block-এ একই দুই লাইন তিনবার লেখা হয়েছে। এটি পুনরাবৃত্তি, এবং অধ্যায় ৩-এ শেখা Logical Operator দিয়ে এটিকে একটি শর্তে আনা যাবে।

চালানো এবং প্রত্যাশিত আচরণ

Terminal-এ `student-result-calculator` Folder থেকে চালান:

Terminal

```
node result.js
```

যে মান দেবেন	যা দেখা যাওয়ার কথা
`'Sara', '80', '90', '70'`	Sara: মোট 240, গড় 80.00, Grade A+, উত্তীর্ণ
`'Rafi', '100', '100', '20'`	Rafi: মোট 220, গড় 73.33, Grade F, অনুত্তীর্ণ
`'Mina', '75', '', '60'`	ইংরেজির নম্বর সঠিক নয়।
কোনো নম্বর `'80.5'`, `'abc'`, `'-5'` বা `'101'`	সংশ্লিষ্ট বিষয়ের নম্বর সঠিক নয়।
নম্বরের আগে-পরে ফাঁকা জায়গা, যেমন `' 80 '`	স্বাভাবিকভাবে গৃহীত

Companion Source Code-এ এই Project-এর ১৪টি Automated Test আছে। সেগুলো চালানোর Command `README.md`-তে দেওয়া আছে।

সমস্যা হলে যা দেখবেন

লক্ষণ	সম্ভাব্য কারণ	করণীয়
মোট নম্বর `8090`-এর মতো জোড়া লাগা	`toMark` ছাড়া সরাসরি যোগ করা হয়েছে	তিনটি নম্বরই `toMark` দিয়ে রূপান্তর করুন
ফাঁকা নম্বর শূন্য হিসেবে গৃহীত	`clean === ''` যাচাইটি নেই বা `Number()`-এর পরে আছে	যাচাইটি রূপান্তরের আগে রাখুন
গড় `NaN`	কোনো নম্বর যাচাই ছাড়া হিসাবে ঢুকেছে	`=== null` যাচাই তিনটি আছে কি না দেখুন
সব সময় `F` দেখায়	`<` এবং `>` উল্টে গেছে	`bangla < PASS_MARK` ঠিক আছে কি না দেখুন
`TypeError: Assignment to constant variable.`	`grade` বা `status` `const` দিয়ে তৈরি	`let` ব্যবহার করুন

এই Project-এর সীমাবদ্ধতা

এটি একটি শিক্ষামূলক Project। নম্বরগুলো Code-এর ভেতরেই লেখা; ব্যবহারকারীর কাছ থেকে নেওয়া হয় না, এবং কোনো ফলাফল সংরক্ষণ করা হয় না। বিষয়ের সংখ্যা তিনটিতে স্থির, কারণ তালিকা নিয়ে কাজ করার উপায় (Array এবং Loop) এখনো শেখানো হয়নি; অধ্যায় ৪ ও ৬-এর পর যেকোনো সংখ্যক বিষয় সামলানো যাবে। শিক্ষার্থীর নাম যাচাই করা হয় না (অনুশীলনী ১২ দেখুন)।

২.১৪ অনুশীলনী

প্রতিটি অনুশীলনী এই অধ্যায় এবং অধ্যায় ১-এর বিষয় দিয়েই সমাধান করা যায়।

প্রাথমিক স্তর

১. তিনটি Variable তৈরি করুন: আপনার নাম, বয়স এবং আপনি শিক্ষার্থী কি না (`true` বা `false`)। প্রতিটির `typeof` দেখান।

২. না চালিয়ে বলুন এই তিনটি কী দেখাবে: `'10' + 5`, `'10' - 5` এবং `10 + 5 + '5'`। তারপর চালিয়ে মিলিয়ে নিন।

৩. নিচের Code-এ একটি ভুল আছে। ভুলটি ঠিক করুন, যাতে `50` দেখায়।

fix-total.js · JavaScript — চলবে: Node.js

```
const total = 0;
total = total + 50;
console.log(total);
```

৪. একটি Variable মান ছাড়া তৈরি করুন এবং আরেকটিতে `null` রাখুন। দুটির মান দেখান এবং পার্থক্যটি নিজের ভাষায় লিখুন।

৫. না চালিয়ে বলুন: `Number('')`, `Number(' ')` এবং `Number('12a')` কী দেবে। তারপর চালিয়ে দেখুন।

মধ্যম স্তর

৬. `toAge` নামে একটি Function লিখুন, যা Text নেয়। Text-টি ০ থেকে ১২০-এর মধ্যে পূর্ণসংখ্যা হলে সেই সংখ্যা ফেরত দেবে; অন্যথায় `null`।

৭. একটি Object তৈরি করে দুটি Variable-এ রাখুন। একটির মাধ্যমে তথ্য বদলে দেখান যে অন্যটিতেও বদলেছে। তারপর `...` দিয়ে আলাদা Copy তৈরি করে দেখান যে এবার মূলটি বদলায় না।

৮. `0.1 + 0.2 === 0.3` কেন `false`, তা ব্যাখ্যা করুন। একই হিসাব পয়সায় (পূর্ণসংখ্যায়) করে দেখান যে তুলনাটি তখন `true` হয়।

৯. `250 / 3`-এর ফল এক দশমিক ঘরে দেখান এবং ফলটির `typeof` দেখান। ফলটির সঙ্গে `1` যোগ করলে কী হয় এবং কেন?

চ্যালেঞ্জ স্তর

১০. **চতুর্থ বিষয়:** চারটি নম্বর (সংখ্যা) নিয়ে মোট ও গড় ফেরত দেয়, এমন একটি Function লিখুন। গড় দুই দশমিক ঘরে দেখান।

১১. **পাস নম্বর:** পাস নম্বরকে একটি নামযুক্ত `const`-এ রেখে `isPassed` Function লিখুন। পাস নম্বর ৩৩ থেকে ৪০ করলে কোন ফল বদলায়, তা দেখান।

১২. **নাম যাচাই:** `cleanName` Function লিখুন, যা নামের আগে-পরের ফাঁকা জায়গা বাদ দেয়, এবং নাম ফাঁকা হলে `null` ফেরত দেয়।

২.১৫ অনুশীলনীর সমাধান

একই সমস্যার একাধিক সঠিক সমাধান থাকতে পারে।

সমাধান ১

about-me.js · JavaScript — চলবে: Browser Console ও Node.js

```
const myName = 'Tanvir';
const age = 21;
const isStudent = true;

console.log(typeof myName);
console.log(typeof age);
console.log(typeof isStudent);
```

Output

```
string
number
boolean
```

সমাধান ২ — যথাক্রমে `105`, `5` এবং `155`। প্রথমটিতে `+`-এর এক পাশে String থাকায় জোড়া লেগেছে। দ্বিতীয়টিতে `-` String-কে সংখ্যায় রূপান্তর করেছে। তৃতীয়টিতে আগে `10 + 5` হয়ে `15`, তারপর `15 + '5'` জোড়া লেগে `'155'`।

সমাধান ৩ — `total`-এর মান বদলায়, তাই `const` নয়, `let` প্রয়োজন।

fix-total.js · JavaScript — চলবে: Browser Console ও Node.js

```
let total = 0;
total = total + 50;
console.log(total);
```

Output

```
50
```

সমাধান ৪

empty-values.js · JavaScript — চলবে: Browser Console ও Node.js

```
let phone;
const fax = null;

console.log(phone);
console.log(fax);
```

Output

```
undefined
null
```

`phone`-এ কোনো মান দেওয়া হয়নি, তাই JavaScript নিজে `undefined` দিয়েছে। `fax`-এ ইচ্ছা করে `null` রাখা হয়েছে, যার অর্থ "Fax নম্বর নেই"।

সমাধান ৫ — যথাক্রমে `0`, `0` এবং `NaN`। ফাঁকা String এবং শুধু ফাঁকা জায়গার String—দুটোই শূন্য হয়, কারণ `Number()` আগে-পরের ফাঁকা জায়গা বাদ দেয়। `'12a'` পুরোটা সংখ্যা নয়, তাই `NaN`।

সমাধান ৬

to-age.js · JavaScript — চলবে: Browser Console ও Node.js

```
function toAge(text) {
  const clean = text.trim();

  if (clean === '') {
    return null;
  }

  const age = Number(clean);

  if (Number.isInteger(age) === false) {
    return null;
  }

  if (age < 0) {
    return null;
  }

  if (age > 120) {
    return null;
  }
```

```
  return age;
}

console.log(toAge(' 25 '));
console.log(toAge('২৫ বছর'));
console.log(toAge('150'));
console.log(toAge(''));
```

Output

```
25
null
null
null
```

সমাধান ৭

shared-object.js · JavaScript — চলবে: Browser Console ও Node.js

```
const book = { title: 'JavaScript Mastery' };
const sameBook = book;

sameBook.title = 'JS Mastery';
console.log(book.title);

const separateBook = { ...book };
separateBook.title = 'অন্য বই';
console.log(book.title);
```

Output

```
JS Mastery
JS Mastery
```

প্রথমবার `book.title` বদলেছে, কারণ `sameBook` একই Object-এর Reference। দ্বিতীয়বার বদলায়নি, কারণ `separateBook` একটি নতুন Object।

সমাধান ৮ — Computer `0.1` এবং `0.2` হুবহু রাখতে পারে না, তাই যোগফল `0.30000000000000004` হয়। পূর্ণসংখ্যায় এই সমস্যা নেই।

paisa.js · JavaScript — চলবে: Browser Console ও Node.js

```
console.log(0.1 + 0.2 === 0.3);
console.log(10 + 20 === 30);
```

Output

```
false
true
```

সমাধান ৯

one-decimal.js · JavaScript — চলবে: Browser Console ও Node.js

```js
const result = (250 / 3).toFixed(1);

console.log(result);
console.log(typeof result);
console.log(result + 1);
```

Output

```
83.3
string
83.31
```

`toFixed()` String ফেরত দেয়, তাই `+ 1` যোগ না করে জোড়া লাগিয়েছে। হিসাব সংখ্যায় শেষ করে তারপর `toFixed()` ব্যবহার করতে হয়।

সমাধান ১০

four-subjects.js · JavaScript — চলবে: Browser Console ও Node.js

```js
function summarize(bangla, english, math, science) {
  const total = bangla + english + math + science;
  const average = total / 4;

  return `মোট ${total}, গড় ${average.toFixed(2)}`;
}

console.log(summarize(80, 90, 70, 85));
```

Output

```
মোট 325, গড় 81.25
```

সমাধান ১১

pass-mark.js · JavaScript চলবে: Browser Console ও Node.js

```
const PASS_MARK = 40;

function isPassed(mark) {
  if (mark >= PASS_MARK) {
    return true;
  }

  return false;
}

console.log(isPassed(35));
console.log(isPassed(40));
```

Output

```
false
true
```

পাস নম্বর ৩৩ থাকলে `isPassed(35)` হতো `true`। সংখ্যাটি এক জায়গায় নাম দিয়ে রাখায় শুধু একটি লাইন বদলাতে হয়েছে।

সমাধান ১২

clean-name.js · JavaScript চলবে: Browser Console ও Node.js

```
function cleanName(text) {
  const clean = text.trim();

  if (clean === '') {
    return null;
  }

  return clean;
}

console.log(cleanName('  Sara  '));
console.log(cleanName('     '));
```

Output

```
Sara
null
```

২.১৬ MCQ: নিজেকে যাচাই করুন

প্রশ্ন ১. নতুন Code-এ Variable তৈরির জন্য প্রথম পছন্দ কোনটি হওয়া উচিত?

ক) `var`

খ) `let`

গ) `const`

ঘ) যেকোনো একটি; কোনো পার্থক্য নেই

প্রশ্ন ২. `typeof null`-এর ফল কী?

ক) `object`

খ) `null`

গ) `undefined`

ঘ) `number`

প্রশ্ন ৩. `Number('')`-এর ফল কী?

ক) `NaN`

খ) `undefined`

গ) `null`

ঘ) `0`

প্রশ্ন ৪. `'5' + 2`-এর ফল কী?

ক) `7`

খ) `'52'`

গ) `NaN`

ঘ) একটি Error

প্রশ্ন ৫. একটি Variable তৈরি করা হয়েছে, কিন্তু কোনো মান দেওয়া হয়নি। তার মান কী?

ক) `null`

খ) `0`

গ) `undefined`

ঘ) ফাঁকা String

প্রশ্ন ৬. কোনো মান `NaN` কি না যাচাইয়ের সঠিক উপায় কোনটি?

ক) `value === NaN`

খ) `Number.isNaN(value)`

গ) `typeof value === 'NaN'`

ঘ) `value == 'NaN'`

প্রশ্ন ৭. `0 === '0'`-এর ফল কী এবং কেন?

ক) `true`, কারণ মান একই

খ) `true`, কারণ String সংখ্যায় রূপান্তরিত হয়

গ) `false`, কারণ শূন্য কোনো কিছুর সমান নয়

ঘ) `false`, কারণ Type আলাদা

প্রশ্ন ৮. `const a = { n: 1 }; const b = a; b.n = 2;` চালানোর পর `a.n`-এর মান কত?

ক) `2`

খ) `1`

গ) `undefined`

ঘ) Error হবে, কারণ `a` একটি `const`

প্রশ্ন ৯. `(10 / 4).toFixed(1)`-এর ফলের Type কী?

ক) `number`

খ) `bigint`

গ) `string`

ঘ) `boolean`

প্রশ্ন ১০. `{ }`-এর ভেতরে `let` দিয়ে তৈরি Variable কোথা থেকে ব্যবহার করা যায়?

ক) পুরো File থেকে

খ) শুধু সেই Block-এর ভেতর থেকে

গ) শুধু Block-এর বাইরে থেকে

ঘ) শুধু Function-এর ভেতর থেকে

২.১৭ MCQ-এর উত্তর ও ব্যাখ্যা

দ্রুত উত্তরমালা: ১–গ · ২–ক · ৩–ঘ · ৪–খ · ৫–গ · ৬–খ · ৭–ঘ · ৮–ক · ৯–গ · ১০–খ

প্রশ্ন	উত্তর	ব্যাখ্যা
১	গ	`const` ভুল করে মান বদলে যাওয়া আটকায়। মান বদলানোর প্রয়োজন হলে তবেই `let`।
২	ক	এটি ভাষার একটি পুরোনো ত্রুটি। `null` যাচাই করতে `=== null` ব্যবহার করতে হয়।
৩	ঘ	ফাঁকা String শূন্য হয়ে যায়। তাই রূপান্তরের আগে ফাঁকা কি না যাচাই জরুরি।
৪	খ	`+`-এর এক পাশে String থাকলে অন্য পাশটিও String হয়ে জোড়া লাগে।
৫	গ	মান না দিলে JavaScript নিজে `undefined` দেয়। `null` Programmer ইচ্ছা করে দেন।
৬	খ	`NaN` নিজের সমানও নয়, তাই `===` দিয়ে ধরা যায় না। `typeof NaN` হলো `number`।
৭	ঘ	`===` Type এবং মান দুটোই মেলায়। `number` এবং `string` আলাদা Type।
৮	ক	`b` একই Object-এর Reference ধরে আছে। `const` Object-এর ভেতরের তথ্য বদলানো আটকায় না।
৯	গ	`toFixed()` দেখানোর জন্য একটি String ফেরত দেয়।
১০	খ	`let` এবং `const` Block-এর ভেতরেই কার্যকর। `var` এই নিয়ম মানে না।

২.১৮ অধ্যায়ের সারসংক্ষেপ

- Variable হলো মানের নাম। প্রথমে `const`; মান বদলালে `let`; নতুন Code-এ `var` নয়।
- `let` ও `const` নিজের Block-এর ভেতরে কার্যকর, এবং Declaration-এর আগে ব্যবহার করা যায় না।
- সাতটি Primitive Type: `string`, `number`, `boolean`, `undefined`, `null`, `bigint`, `symbol`। বাকি সব Object।
- `typeof` মানের Type জানায়; ব্যতিক্রম `typeof null`, যা `object` দেখায়।
- দশমিক হিসাব সব সময় নিখুঁত নয়; `NaN` ধরতে `Number.isNaN()`।
- `undefined` মানে মান দেওয়া হয়নি; `null` মানে ইচ্ছাকৃতভাবে কিছু নেই।
- Type থাকে মানের, Variable-এর নয়; তবু একটি Variable-এ একটিই Type রাখা উচিত।
- নিজে রূপান্তর করুন, এবং `Number('')` যে `0` দেয় তা মনে রাখুন।
- সব সময় `===` ও `!==`।
- Primitive-এ মান Copy হয়; Object-এ ঠিকানা (Reference) Copy হয়। `const` Object-এর ভেতরের তথ্য বদলানো আটকায় না।

পরের অধ্যায়ে Operator এবং সিদ্ধান্ত গ্রহণ: হিসাব, তুলনা, একাধিক শর্ত জোড়া লাগানো এবং `if`, `else`, `switch` দিয়ে Program-কে ভিন্ন পথে চালানো।

২.১৯ গুরুত্বপূর্ণ পরিভাষা

পরিভাষা	অর্থ
Declaration	একটি Variable তৈরি করা।
Assignment / Reassignment	Variable-এ মান রাখা / নতুন মান রাখা।
Block	`{ }`-এর ভেতরের Code।
Scope	Program-এর যে অংশ থেকে একটি Variable ব্যবহার করা যায়।
Data Type	মানের ধরন, যা ঠিক করে মানটি নিয়ে কী করা যাবে।
Primitive	একটি সরল মান: string, number, boolean, undefined, null, bigint, symbol।
Object	একাধিক তথ্য একসঙ্গে রাখার মান।
Dynamic Typing	Type মানের সঙ্গে থাকে; একই Variable-এ ভিন্ন Type-এর মান রাখা যায়।
Type Conversion	এক Type-এর মানকে অন্য Type-এ বদলানো।
Explicit / Implicit Conversion	নিজে করা রূপান্তর / JavaScript-এর নিজে থেকে করা রূপান্তর।
NaN	Not a Number: অর্থহীন সংখ্যা-হিসাবের ফল।
Strict Equality	`===` : Type এবং মান দুটোই মেলায়।
Reference	মেমোরিতে একটি Object-এর ঠিকানা।
Garbage Collection	অব্যবহৃত Object-এর মেমোরি স্বয়ংক্রিয়ভাবে ফিরিয়ে নেওয়া।

২.২০ আরও পড়ুন

- **MDN — Grammar and types:** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Grammar_and_types` — Variable ও Type-এর পূর্ণ বিবরণ।
- **MDN — JavaScript data types and data structures:** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Data_structures` — প্রতিটি Type-এর বিস্তারিত।
- **MDN — typeof:** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/typeof` — `typeof`-এর সব সম্ভাব্য ফল।
- **MDN — Equality comparisons and sameness:** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Equality_comparisons_and_sameness` — `==` এবং `===`-এর তুলনা।